

Cafe-Searching in Buenos Aires — An Expert System

Minerva University

CS152 - Harnessing Artificial Intelligence Algorithms

Dec 15th, 2025

Introduction

One of the most apparent needs for Minerva students is finding cafés that fit their study routines and constraints. Yet in a new rotation city, discovering a place that is quiet enough, has reliable Wi-Fi and outlets, is close to the residence, and is affordable often involves a lot of trial and error. Once we finally find somewhere that “works well enough,” we tend to cling to it and stop exploring other options, which can hinder further engagement with the city.

In order to address these challenges, this project designs and implements an expert system in Prolog that recommends cafés near the residence hall in Buenos Aires based on students’ preferences. By answering a short sequence of menu-style questions, users can quickly narrow down a list of 23 nearby cafés to those that best match their desired distance, budget, outlet availability, noise level, crowding, menu, closing time, Wi-Fi quality, and size.

System Focus and Askables

Our expert system focuses on recommending cafés near the Minerva residence in Buenos Aires for everyday student use, such as studying, meeting friends, or working. We restrict the knowledge base to places that at least one group member has already visited, ensuring the information is accurate.

We identified nine key attributes (askables) that matter for most Minerva students when choosing a café:

1. Distance from residence (<1 km, 1–2 km, 2–3 km, >3 km)
2. Budget (under 10k, 10k–20k, no preference)
3. Outlets availability (yes, no, no preference)
4. Noise level (low, mid, high, no preference)
5. Crowdedness (rarely crowded, sometimes crowded, usually crowded, no preference)
6. Goods offered (just coffee, coffee and pastries, full menu, no preference)
7. Closing time (closes before 7 PM, closes between 7–9 PM, closes after 9 PM, no preference)
8. Wi-Fi quality (low, mid, high)
9. Size (small, medium, large)

Data Collection

All data for the cafés was collected from Google Maps and verified through personal experience:

- Each group member contributed 5–6 cafés that they had personally visited at least once.
- In total, our group compiled information for 23 cafés near the residence hall.

For each café, we recorded objective attributes (taken primarily from Google Maps or café websites), such as typical price range, opening hours, and whether they offer only drinks or a fuller food menu. We also collected subjective attributes based on our own experiences, including noise level, crowdedness, Wi-Fi quality, and perceived size. All of this information was stored in a shared [spreadsheet](#), which then guided the creation of Prolog facts.

Expert System Logic and Decision Tree

The expert system follows a sequential filtering strategy. It first prompts the user for their preferred distance and filters the full list of cafés to only those that match the selected distance. Next, it asks for budget and further narrows the remaining cafés based on the user's answer. This same logic is then applied to each askable in a fixed order. At each step, the user's choice adds an additional constraint to the current candidate set. When "no preference" is available and selected, that attribute is simply skipped and does not restrict the results. Once all askables have been processed, if no cafés satisfy all of the specified criteria, the system returns none; if one or more cafés match, it returns the full list of cafés that meet the user's preferences.

We visualized this logic as a decision tree. Starting from the root node representing all cafés, each level corresponds to one askable, and each branch represents one of the possible answers for that attribute. This visualization, shown in our [Figma diagram](#), illustrates how different combinations of preferences lead to different subsets of cafés. For clarity, we chose to terminate branches in the diagram once a single café remains.

Implementation

Our expert system is implemented using the pyswip library. Prolog is responsible for storing the knowledge base and performing logical inference, while Python handles the user interface (natural-language questions, menu selection, input parsing, and printing results).

Overall architecture:

- We define the Prolog knowledge base (KB) as a multiline string in Python, with each café encoded as a single `cafe/10` fact.
- Python writes this string to a temporary `.pl` file and calls `prolog.consult(kb_path)` to load it.
- At runtime, Python:

1. clears any previous state with `prolog.retractall("known(_,_,_))",`
2. iterates through all askables in a fixed order, calling `ask_attr` for each,
3. after each answer, asserts the corresponding `known/3` facts and uses `matches_criteria/1` to incrementally filter the set of possible caf  s, shrinking the set of caf  s that satisfy the constraints so far,
4. once all questions have been asked, prints all caf  s that satisfy the full set of user-selected constraints.

We use a dynamic predicate `known/3` to store user choices at runtime: `known(yes, Attribute, Value)` means the user has accepted that value (and there may be several such facts for the same attribute when the user selects multiple options), while `known(no_preference, Attribute, none)` indicates that the user does not want that attribute to constrain the results. The helper predicate `match_attr(Attribute, CafeValue)` implements the matching logic by allowing any value when there is a “no preference” fact, requiring exact equality when a specific value has been chosen, and ignoring attributes that have not been set at all. The main predicate `matches_criteria(Name)` unpacks all attributes of a caf  , applying `match_attr/2` to each, and succeeds only if all attributes are compatible with the current known facts.

On the Python side, we define two helper functions, `assert_yes(attr, value)` and `assert_no_preference(attr)`, which assert the corresponding `known/3` facts into Prolog. The `MENUS` dictionary maps each attribute (e.g., 'distance', 'goods') to a list of (label, `prolog_atom`) pairs, such as "Less than 1 km" $\rightarrow$ `less_than_1km`". To make the system more natural to use, we also maintain a `QUESTIONS` dictionary with short, human-readable prompts for each attribute (e.g., “How far are you willing to walk to the cafe?”). A `MULTISELECT` set marks attributes where it makes sense to allow multiple acceptable categories, while ‘outlets’ remains single-select. The function `ask_attr(attr)` prints the question, lists the numbered options plus a “No preference” option, and reads user input. For multi-select attributes, it accepts comma-separated choices (e.g., 1,3) and asserts a `known(yes, Attribute, Value)` fact for each valid selection; for single-select attributes, it asserts a single yes fact based on the first valid number or a `no_preference` fact if the user skips. After each call to `ask_attr`, `main()` runs a quick Prolog query to `matches_criteria(Name)` (limited to one result) to check whether any caf   is still compatible; if none are, it exits early and tells the user their combination is too restrictive. If at least one caf   matches all askables, `main()` performs a final query for all matching names and prints them as a numbered list.

Refer to [Appendix B](#) for the full implementation.

Testing

We tested the system using three realistic student scenarios:

Test Case 1 – Quiet café, close by, cheap:

Goal: A small café within 1 km, inexpensive, quiet, with good enough Wi-Fi, offering only coffee and pastries, and open until at least 7 PM. The full transcript of the interaction for this test case is included in Appendix B.

System Output:

- Posdata
- Divino Budin

Both recommendations align well with our expectations: they are close to the residence, reasonably priced, and generally quiet spaces suitable for studying with coffee and pastries. We also verified that this outcome is consistent with the corresponding branch of our Figma decision tree, confirming that both the implementation and the logic visualization are in agreement.

Test Case 2 – Late-night study place, large, full menu

Goal: A larger café suitable for group work, with a full menu and good Wi-Fi, open late into the evening.

System output

- Lobo Café
- Amayta Patisserie

Both cafés are known for their larger spaces, extensive menus, and late closing times, making them ideal for group study sessions or longer stays.

Test Case 3 — Large study cafe far from the residence hall

Goal: A spacious cafe with good food, Wi-Fi and outlets, can be farther away from the residence hall.

System output

- Kopi Café

- Casa Dingo

Both outputs match this profile well because they are located in a farther neighbourhood, offer a full menu, and provide reliable Wi-Fi and outlets.

Group Contributions

- Emma: Contributed to the brainstorming of topic and askables, made the logic tree (and the associated write-up sections), edited/reviewed the final paper before submitting.
- Anhelina: Contributed to the topic selection, wrote the Prolog code, and menu based extension.
- Dasha: Contributed to the project management and the structuring of the paper, reviewed the askables to the Prolog-specific format, as well as the test case design.
- Maria: Contributed to structuring and writing the final report and incorporating the natural-language questions used in the Python interface

Appendix A: HC/LOs

#rightproblem: While picking a topic, we brainstormed our biggest challenges in the city, and concluded that finding coffee shops/cafes suitable for Minerva work is difficult because it requires specific attributes such as wifi quality (high quality required to take classes), low noise levels (for focused study sessions) and closing time (as Minervans tend to work late). Along with these, we listed out the most important factors we consider when choosing a cafe to visit, which became our askables. In this way, we ensured that our expert system is not only accurate but also specifically applicable to address problems that Minerva students face, drawn from our own experiences.

#evidencebased: Throughout the expert system, we firstly backed our askables with personal evidence, because as Minervans, we know the most important qualities Minerva students look for in a cafe, and we also ensured that our collected data on cafes was correct (see #sourcequality for methods). Finally, we performed one test case and visited the cafe as a group (Kopi Cafe).

#sourcequality: We used Google Maps, customer reviews, and individual cafe websites to collect data and ensure accuracy. For volatile attributes, such as price (as Argentina's economy is unstable), we ensured that we referenced recent data (less than a month). Between the four of us, we visited every cafe on our list, adding an additional layer of insurance that the data is up-to-date and correct.

#ailogic: Our project structures the café recommendation system as a rule-based inference engine. Each café is represented as a Prolog fact with nine attributes, and the system reasons over these facts using the predicate 'matches_criteria/1' that enforces all relevant constraints. Instead of checking all attributes manually, the system uses the general predicate 'match_attr/2' to determine whether a café satisfies a user preference, allowing the logic to generalize across all askables without writing separate rules for each attribute. Our decision-tree visualization reflects this exact reasoning process, showing how each attribute prunes the hypothesis space until a small set of cafés remains. This demonstrates AI logic by using symbolic representations, declarative rules, and backward reasoning through Prolog queries.

#aicoding: We implemented the system using a Python-Prolog interface via pyswip. Python manages the user interaction through menu-based prompts, while Prolog handles inference. Each answer dynamically updates the knowledge base through calls like 'assertz(known(yes, A, V))'. At runtime, the Python script writes our KB into a temporary '.pl' file, consults it, and queries Prolog for matching cafés. The menu interface is an extension beyond the class example, since users select options from lists rather than typing free-form answers.

#modeling: Our system models real cafés in Buenos Aires by encoding their attributes into Prolog facts and filtering them through user constraints. We tested the model with three realistic student scenarios and confirmed that the recommendations matched the logic of our decision tree. We also visited one recommended café to check if the system's stored attributes matched reality. This combination of structured data, real test cases, and field verification demonstrates that our expert system functions as a simple but effective real-world model.



Image 1. At Kopi Cafe

Test case #3 (see above) recommended Kopi Cafe. We visited Kopi as a group and verified that it met all our conditions:

- Over 3km
- Between 10k-20k
- With outlets
- Mid noise level
- Sometimes_crowded

- Full menu
- Closes _between_7_and_9
- High wifi quality
- Large size

Thus, we determined that our expert system was effective and accurate.

AI Statement

Grammarly was used for surface-level editing.

Note

We have partially reused the code from Session 21 to create an expert system, incorporating additional complexities such as menu-based options and Python for user-facing interaction.

Appendix B: Prolog and Python implementation

```

try:
    import pyswip
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, "-m", "pip", "install",
        "pyswip"])
    import pyswip

import tempfile, os
from pyswip import Prolog, Functor, Atom
from pyswip.prolog import Variable

KB = r'''
% cafes_fixed.pl -- simplified KB for Python-driven menus
:- dynamic known/3.

% Cafe facts (name, distance, budget, outlets, noise_level, crowdedness,
goods, closing_time, wifi_quality, size)
cafe('Amayta Patisserie', between_1_and_2km, between_10k_20k, yes, mid,
sometimes_crowded, full_menu, closes_after_9, high, large).
cafe('Casa Dingo', over_3km, under_10k, yes, mid, sometimes_crowded,
full_menu, closes_between_7_and_9, high, large).
cafe('Casa Telma', between_2_and_3km, between_10k_20k, yes, mid,
usually_crowded, full_menu, closes_between_7_and_9, high, large).
cafe('Divino Budin', less_than_1km, under_10k, no, low, rarely_crowded,
coffee_and_pastries, closes_between_7_and_9, high, small).
cafe('Doxa cafe y vino', between_2_and_3km, between_10k_20k, no, low,
rarely_crowded, full_menu, closes_between_7_and_9, high, small).
cafe('Guaduchi Coffee House', over_3km, between_10k_20k, yes, low,
rarely_crowded, full_menu, closes_between_7_and_9, high, medium).
cafe('Hobby - Cafe de Especialidad', over_3km, between_10k_20k, yes, mid,
sometimes_crowded, coffee_and_pastries, closes_after_9, high, large).
cafe('Kopi Cafe', over_3km, between_10k_20k, yes, mid, sometimes_crowded,
full_menu, closes_between_7_and_9, high, large).
cafe('Lharmonie', over_3km, between_10k_20k, yes, mid, rarely_crowded,
full_menu, closes_after_9, mid, medium).
cafe('Llama Coffee Roasters', between_1_and_2km, between_10k_20k, yes, low,
sometimes_crowded, just_coffee, closes_between_7_and_9, mid, large).
cafe('Lobo Café Puerto Madero', between_2_and_3km, between_10k_20k, no,
low, sometimes_crowded, full_menu, closes_after_9, high, large).

```

```

cafe('Magnolia's Cafe', over_3km, under_10k, no, low, rarely_crowded,
full_menu, closes_between_7_and_9, high, small).
cafe('Manifiesto (Retiro)', less_than_1km, under_10k, yes, mid,
usually_crowded, full_menu, closes_between_7_and_9, high, medium).
cafe('Merci', between_2_and_3km, between_10k_20k, yes, low, rarely_crowded,
full_menu, closes_between_7_and_9, mid, medium).
cafe('Moshu Treehouse', over_3km, between_10k_20k, yes, mid,
sometimes_crowded, full_menu, closes_between_7_and_9, mid, large).
cafe('Posdata Cafe Postal', less_than_1km, under_10k, no, low,
rarely_crowded, coffee_and_pastries, closes_between_7_and_9, mid, small).
cafe('Punto Café', less_than_1km, under_10k, yes, mid, sometimes_crowded,
full_menu, closes_between_7_and_9, high, medium).
cafe('Rajatabla cafe', between_1_and_2km, between_10k_20k, yes, mid,
usually_crowded, full_menu, closes_between_7_and_9, high, large).
cafe('Sorbo Cafe', between_2_and_3km, between_10k_20k, yes, mid,
usually_crowded, full_menu, closes_between_7_and_9, high, medium).
cafe('Tostado - Corrientes y Esmeralda', less_than_1km, under_10k, yes,
mid, usually_crowded, full_menu, closes_between_7_and_9, high, large).
cafe('Usina Cafetera Recoleta', between_2_and_3km, between_10k_20k, yes,
mid, usually_crowded, full_menu, closes_between_7_and_9, high, large).
cafe('WAT Coffee', between_2_and_3km, under_10k, yes, mid, rarely_crowded,
coffee_and_pastries, closes_between_7_and_9, high, medium).
cafe('Yen Cafe', over_3km, under_10k, yes, low, rarely_crowded, full_menu,
closes_between_7_and_9, high, medium).

% ----- Matching helper -----
% If user said no preference on attribute A, succeed.
match_attr(A, _) :-
    known(no_preference, A, none), !.

% If user selected one or more yes values, any matching yes value succeeds.
match_attr(A, CafeValue) :-
    known(yes, A, CafeValue), !.

% If attribute hasn't been asked yet (no known fact), succeed (don't filter
on it yet)
match_attr(A, _) :-
    \+known(_, A, _), !.

% Otherwise fail (means this cafe doesn't match user's preferences)
match_attr(_, _) :- fail.

% Top-level matcher: all attributes must match (or be no_preference)

```

```

matches_criteria(Name) :-
    cafe(Name, Dist, Budget, Outlets, Noise, Crowd, Goods, Close, Wifi,
Size),
    match_attr(distance, Dist),
    match_attr(budget, Budget),
    match_attr(outlets, Outlets),
    match_attr(noise_level, Noise),
    match_attr(crowdedness, Crowd),
    match_attr(goods, Goods),
    match_attr(closing_time, Close),
    match_attr(wifi_quality, Wifi),
    match_attr(size, Size).
...

# Write KB to temp file
fd, kb_path = tempfile.mkstemp(suffix='.pl', text=True)
with os.fdopen(fd, 'w') as f:
    f.write(KB)

# Start Prolog and consult KB
prolog = Prolog()
prolog.consult(kb_path)

# Simple utility to assert known facts
def assert_no_preference(attr):
    # store as known(no_preference, Attr, none).
    prolog.assertz(f"known(no_preference, {attr}, none)")

def assert_yes(attr, value):
    # store as known(yes, Attr, Value).
    prolog.assertz(f"known(yes, {attr}, {value})")

# Menu definitions (display_label, prolog_atom)
MENUS = {
    'distance': [
        ("Less than 1 km", "less_than_1km"),
        ("1-2 km", "between_1_and_2km"),
        ("2-3 km", "between_2_and_3km"),
        ("Over 3 km", "over_3km")
    ],
    'budget': [
        ("Under 10k", "under_10k"),
        ("10k-20k", "between_10k_20k"),

```

```

        ("Over 20k", "over_20k")
    ],
    'outlets': [
        ("Yes", "yes"),
        ("No", "no")
    ],
    'noise_level': [
        ("Low", "low"),
        ("Mid", "mid"),
        ("High", "high")
    ],
    'crowdedness': [
        ("Rarely crowded", "rarely_crowded"),
        ("Sometimes crowded", "sometimes_crowded"),
        ("Usually crowded", "usually_crowded")
    ],
    'goods': [
        ("Just coffee", "just_coffee"),
        ("Coffee and pastries", "coffee_and_pastries"),
        ("Full menu", "full_menu")
    ],
    'closing_time': [
        ("Closes before 7 PM", "closes_before_7"),
        ("Closes between 7-9 PM", "closes_between_7_and_9"),
        ("Closes after 9 PM", "closes_after_9")
    ],
    'wifi_quality': [
        ("Low", "low"),
        ("Mid", "mid"),
        ("High", "high")
    ],
    'size': [
        ("Small", "small"),
        ("Medium", "medium"),
        ("Large", "large")
    ]
}

# Natural language questions for each attribute
QUESTIONS = {
    'distance': "How far are you willing to walk to the cafe?",
    'budget': "What's your budget per person?",
    'outlets': "Do you need power outlets?",

```

```

    'noise_level': "What noise level do you prefer?",
    'crowdedness': "How busy do you want the cafe to be?",
    'goods': "What kind of food options do you want?",
    'closing_time': "How late do you need the cafe to be open?",
    'wifi_quality': "What WiFi quality do you need?",
    'size': "What size cafe do you prefer?"
}

# Which attributes are multi-select (user can pick more than one)
MULTISELECT = {'goods', 'distance', 'budget', 'crowdedness', 'noise_level',
               'closing_time', 'wifi_quality', 'size'}

# Query an attribute from the user, assert known facts into Prolog
def ask_attr(attr):
    options = MENUS[attr]
    question = QUESTIONS.get(attr, attr.replace('_', ' ').title())
    print(f"\n{question}")

    # Showing menu options
    for i, (label, atom) in enumerate(options, start=1):
        print(f" {i}. {label}")
    print(f" {len(options)+1}. No preference")

    # prompt
    if attr in MULTISELECT:
        prompt = "Pick option numbers separated by commas (e.g. 1,3) or
press Enter for no preference: "
    else:
        prompt = "Pick one option number or press Enter for no preference: "
    "

    resp = input(prompt).strip()
    if not resp:
        assert_no_preference(attr)
        return

    # parse numbers
    nums = []
    for token in resp.split(','):
        token = token.strip()
        if not token:
            continue
        if not token.isdigit():
            print(f"Ignoring invalid token: {token}")

```

```

        continue
    nums.append(int(token))
if not nums:
    assert_no_preference(attr)
    return
# If user explicitly chose the no-preference number
if any(n == len(options)+1 for n in nums):
    assert_no_preference(attr)
    return
# For single-select attributes, take the first valid number
if attr not in MULTISELECT:
    n = nums[0]
    if 1 <= n <= len(options):
        _, val = options[n-1]
        assert_yes(attr, val)
    else:
        assert_no_preference(attr)
else:
    # multi-select: assert yes for each chosen value
    chosen_any = False
    for n in nums:
        if 1 <= n <= len(options):
            _, val = options[n-1]
            assert_yes(attr, val)
            chosen_any = True
    if not chosen_any:
        assert_no_preference(attr)

def main():
    """Main function to run the cafe recommendation system."""
    print("=" * 60)
    print("Welcome to the Buenos Aires Cafe Finder!")
    print("=" * 60)
    print("\nLet's find the perfect cafe for you.")
    # clear any previous known facts
    prolog.retractall("known(_,_,_)")
    # Ask attributes in a sensible order (you can change)
    order =
['distance', 'budget', 'outlets', 'noise_level', 'crowdedness', 'goods', 'closing
_time', 'wifi_quality', 'size']
    for a in order:
        ask_attr(a)
        # Check for matches after each question - stop early if no matches

```

```

possible
    results = list(prolog.query("matches_criteria(Name).",
maxresult=1))
    if not results:
        print(f"\n No cafes match your criteria so far (after selecting
{a.replace('_', ' ')}).")
        print("Stopping early to save time. Try adjusting your
preferences!")
        return

    # Final query to get all matches
    results = list(prolog.query("matches_criteria(Name).", maxresult=100))
    if results:
        print(f"\nFound {len(results)} matching cafe(s):")
        for i, sol in enumerate(results, start=1):
            name = sol['Name']
            # pyswip may return bytes for atoms on some setups; coerce
            if isinstance(name, bytes):
                name = name.decode('utf-8')
            print(f"{i}. {name}")
    else:
        print("\nNo cafes match your criteria. Try relaxing preferences.")

if __name__ == "__main__":
    try:
        main()
    finally:
        # cleanup kb file
        try:
            os.remove(kb_path)
        except Exception:
            pass

```

Interactions with the Expert System.

Test case 1:

```

=====
Welcome to the Buenos Aires Cafe Finder!
=====

```


Let's find the perfect cafe for you.

How far are you willing to walk to the cafe?

1. Less than 1 km
2. 1-2 km
3. 2-3 km
4. Over 3 km
5. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

1

What's your budget per person?

1. Under 10k
2. 10k-20k
3. Over 20k
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

1

Do you need power outlets?

1. Yes
2. No
3. No preference

Pick one option number or press Enter for no preference:

What noise level do you prefer?

1. Low
2. Mid
3. High
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

1

How busy do you want the cafe to be?

1. Rarely crowded
2. Sometimes crowded
3. Usually crowded
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no

preference:

What kind of food options do you want?

1. Just coffee
2. Coffee **and** pastries
3. Full menu
4. No preference

Pick option numbers separated by commas (e.g. 1,3) **or** press Enter **for** no preference:

2

How late do you need the cafe to be open?

1. Closes before 7 PM
2. Closes between 7-9 PM
3. Closes after 9 PM
4. No preference

Pick option numbers separated by commas (e.g. 1,3) **or** press Enter **for** no preference:

2

What WiFi quality do you need?

1. Low
2. Mid
3. High
4. No preference

Pick option numbers separated by commas (e.g. 1,3) **or** press Enter **for** no preference:

2,3

What size cafe do you prefer?

1. Small
2. Medium
3. Large
4. No preference

Pick option numbers separated by commas (e.g. 1,3) **or** press Enter **for** no preference:

1

```
=====
Searching for cafes that match your preferences...
=====
```

Found 2 cafe(s) that match your criteria:

1. Divino Budin
2. Posdata Cafe Postal

Test Case 2:

Welcome to the Buenos Aires Cafe Finder!

=====

Let's find the perfect cafe for you.

How far are you willing to walk to the cafe?

1. Less than 1 km
2. 1-2 km
3. 2-3 km
4. Over 3 km
5. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

What's your budget per person?

1. Under 10k
2. 10k-20k
3. Over 20k
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

Do you need power outlets?

1. Yes
2. No
3. No preference

Pick one option number or press Enter for no preference:

What noise level do you prefer?

1. Low
2. Mid
3. High
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

How busy do you want the cafe to be?

1. Rarely crowded
2. Sometimes crowded
3. Usually crowded
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

What kind of food options do you want?

1. Just coffee
2. Coffee and pastries
3. Full menu
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 3

How late do you need the cafe to be open?

1. Closes before 7 PM
2. Closes between 7-9 PM
3. Closes after 9 PM
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 3

What WiFi quality do you need?

1. Low
2. Mid
3. High
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 3

What size cafe do you prefer?

1. Small
2. Medium
3. Large
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 3

=====

Searching for cafes that match your preferences...

```
=====
```

Found 2 cafe(s) that match your criteria:

1. Amayta Patisserie
2. Lobo Café Puerto Madero

Test Case 3:

```
Welcome to the Buenos Aires Cafe Finder!
```

```
=====
```

Let's find the perfect cafe for you.

How far are you willing to walk to the cafe?

1. Less than 1 km
2. 1-2 km
3. 2-3 km
4. Over 3 km
5. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 4

What's your budget per person?

1. Under 10k
2. 10k-20k
3. Over 20k
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

Do you need power outlets?

1. Yes
2. No
3. No preference

Pick one option number or press Enter for no preference: 1

What noise level do you prefer?

1. Low
2. Mid
3. High
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

How busy do you want the cafe to be?

1. Rarely crowded
2. Sometimes crowded
3. Usually crowded
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

What kind of food options do you want?

1. Just coffee
2. Coffee and pastries
3. Full menu
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 3

How late do you need the cafe to be open?

1. Closes before 7 PM
2. Closes between 7-9 PM
3. Closes after 9 PM
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference:

What WiFi quality do you need?

1. Low
2. Mid
3. High
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 3

What size cafe do you prefer?

1. Small
2. Medium
3. Large
4. No preference

Pick option numbers separated by commas (e.g. 1,3) or press Enter for no preference: 3

```
=====
Searching for cafes that match your preferences...
=====
```

Found 2 cafe(s) that match your criteria:

1. Casa Dingo
2. Kopi Cafe